

Présentation INVOKE - Version Technique

Durée : 10 minutes

1. Introduction Technique (1 minute)

INVOKE - Plateforme JDR

- **Concept** : Mise en relation joueurs/maîtres de jeu avec stack moderne
- **Focus présentation** : Architecture, méthodologie et innovations techniques
- **État actuel** : Projet en développement continu
- **Staging** : Version avancée avec nouvelles fonctionnalités vs production

Points clés à mentionner

- Application fonctionnelle en production
- Architecture évolutive et scalable
- Approche technique moderne

2. Architecture Multi-Environnements (2.5 minutes)

Gestion des Environnements

- **Local** : Développement avec hot-reload, debugging complet
- **Staging** : Version avancée, laboratoire d'innovation
- **Production** : Version stable et validée

Configuration .env par Environnement

```
# Local
DATABASE_URL=mongodb://localhost:27017/invoke-local
API_URL=http://localhost:3000
NODE_ENV=development

# Staging
DATABASE_URL=mongodb+srv://staging.cluster.mongodb.net
API_URL=https://invoke-staging.render.com
NODE_ENV=staging

# Production
DATABASE_URL=mongodb+srv://prod.cluster.mongodb.net
```

```
API_URL=https://invoke-api.render.com
NODE_ENV=production
```

CI/CD Pipeline

- **GitHub** : Gestion de versions et branches
- **Render** : Auto-déploiement selon la branche
- **Workflow** : Push → Build → Deploy automatique

3. Architecture Technique Avancée (2.5 minutes)

Stack Technologique

- **Frontend** : React 18 + TypeScript + Vite + Tailwind CSS
- **Backend** : Node.js + Express + Mongoose
- **Base de données** : MongoDB Atlas (géo-distribué)
- **Déploiement** : Render (frontend + backend)

API RESTful Modulaire

10+ modules organisés :

- `auth` : Authentification JWT + cookies sécurisés
- `users` : Gestion profils MJ/joueurs + avatars
- `sessions` : CRUD parties + inscriptions
- `campaigns` : Regroupement sessions longue durée
- `games` : Catalogue jeux + mise en avant
- `feedback` : Système d'évaluations
- `conversations` : Messagerie intégrée
- `upload` : Gestion centralisée des images

Sécurité Avancée

- **JWT** : Tokens signés avec expiration + refresh
- **CORS** : Whitelist dynamique des domaines autorisés
- **Rate Limiting** : Protection DoS par IP/route
- **RBAC** : Contrôle d'accès basé sur les rôles
- **Validation** : Schémas Joi côté serveur + client

4. Tests & Assurance Qualité (2 minutes)

Stratégie de Tests

- **Tests API** : Validation systématique endpoints en début de développement
- **Collections Postman** : Scénarios automatisés
- **Tests manuels outillés** : Scripts Node.js dédiés
- **TypeScript** : Compilation stricte = premier filet de sécurité

Qualité de Code

- **Linting** : ESLint + Prettier pour standards homogènes
- **Error Handling** : Middleware centralisé avec classe `AppError`
- **Logging** : Winston avec rotation quotidienne
- **Git Hooks** : Validation pre-commit

Monitoring

- **Health Check** : Route `/api/v1/health` pour monitoring
- **Logs structurés** : Traçabilité complète des erreurs
- **Performance** : Suivi temps de réponse API

5. Fonctionnalités Techniques Clés (1.5 minute)

Système d'Upload Avancé

- **Multi-types** : Avatars, bannières sessions/campagnes, images jeux
- **Validation** : Taille, format, sécurité
- **Stockage** : Organisation par type et ID utilisateur
- **API unifiée** : Routes centralisées pour tous les uploads

Recherche & Filtrage

- **Filtres multiples** : Jeux, dates, niveau, format (IRL/en ligne)
- **Pagination** : Performance sur gros volumes
- **Tri** : Date, popularité, évaluations
- **Cache** : Optimisation requêtes fréquentes

Dashboard Dynamique

- **Statistiques temps réel** : Sessions, utilisateurs, évaluations
- **Rôles adaptatifs** : Interface différente MJ/joueurs/admin
- **Graphiques** : Évolution activité, répartition jeux
- **Actions rapides** : Création session, modération

Architecture Messagerie

- **Structure prête** : Conversations + participants
- **WebSocket ready** : Architecture préparée temps réel
- **Modération** : Interface admin intégrée
- **Notifications** : Système extensible

6. Méthodologie & Workflow Git (1 minute)

Stratégie Prévue vs Réalité

Workflow Planifié :

```
[Feature] → [Dev] → [Staging] → [Production]
```

Workflow Adapté :

```
[Feature] → [Staging] → [Production]
```

Défis & Adaptations

- **Challenge** : Application stricte du workflow en projet solo
- **Pragmatisme** : Adaptation pour maintenir la vélocité
- **Résultats** : 3 environnements fonctionnels malgré simplification
- **Apprentissage** : Importance discipline Git même en solo

Innovations Méthodologiques

- **IA comme copilote** : Debugging, solutions alternatives, documentation auto
- **Rétros avec IA** : Synthèse automatique des sessions de travail
- **Journaling** : Documentation continue dans `/docs`
- **Agile adapté** : Sprints courts avec backlog évolutif

7. Points de Dépassement & Évolutions (0.5 minute)

Implémentations Bonus Possibles

- **Swagger** : Documentation interactive (30min d'implémentation)
- **Tests E2E** : Cypress/Playwright sur parcours critiques
- **WebSocket** : Chat temps réel (architecture prête)
- **Monitoring** : Sentry pour tracking erreurs production

Roadmap Technique

- **Microservices** : Migration progressive par module
- **Mobile** : React Native avec API existante
- **Serverless** : Functions pour tâches spécifiques
- **CDN** : Optimisation assets statiques

Métriques Actuelles

- **Performance** : API < 300ms sur requêtes simples
- **Disponibilité** : 99%+ grâce MongoDB Atlas + Render
- **Scalabilité** : Architecture prête 1000+ utilisateurs simultanés

Conclusion - Démonstration

Points forts à démontrer :

1. **Multi-environnements** : Montrer staging vs production
2. **API complète** : Tests en direct via interface
3. **UX responsive** : Mobile + desktop
4. **Dashboard admin** : Statistiques temps réel
5. **Upload system** : Avatars + bannières fonctionnels

Questions attendues :

- Architecture scalabilité
- Choix technologiques
- Gestion des erreurs
- Sécurité implémentée
- Tests & validation

Projet vivant en évolution continue !

